

When the Planner's Misestimation Inflates Your Cloud Bill

Finding the Costly Queries `pg_stat_statements` Can't See

Andrei Lepikhov

pgEdge

PGDay Israel, 25.10.2026



From Michael B. Williams (michael.williams@glexia.com)
Subject **Segmentation fault caused by Postgrest - reateplan.c:6178 - prepare_sort_from_pathkeys**

2023-12-31 03:23:12.048 PST [17763] LOG: server process (PID 19014) was terminated by **signal 11: Segmentation fault**

2023-12-31 03:23:12.048 PST [17763] DETAIL: Failed process was running:

with recursive

pks_fks as (

-- pk + fk referencing col

select

contype::text as contype,

conname,

array_length(conkey, 1) as ncol,

conrelid as resorigtbl,

col as resorigcol,

ord

from pg_constraint

left join lateral unnest(conkey) with ordinality as _(col, ord)

where contype IN ('p', 'f')

:
: and then the whole gdb session: #0 ... #21



private chat with an AQO user

12 Jan 2024

Be careful using AQO – just remember, AI guesses, and these predictions can sometimes be totally wrong. The best results I achieved were when I used it manually, correcting only specific problematic queries. 1:41 ✓✓

We have turned on AQO across **5 000 servers** for all queries now 3:06

We love it. 3:06

Do you have some positive effect? 3:07 ✓✓

Yes. 3:15

Much faster across the board. 3:15

I.e. run the query 50% time without AQO then 50% w/ AQO then compare 3:18



The **query plan** is a great source of savings on your cloud bill.

It requires no extra hardware or watts — just a little thought and careful tuning.



Is it easy to spot an optimisation problem?

```
SELECT T1.Fid100343RRef,
       (CAST(CAST((T16.Fid100345_ * T1.Fid100346_) AS NUMERIC(28,8)) /
              (T16.Fid100346_ * T1.Fid100345_) AS NUMERIC(28,8)))
FROM (SELECT T15.Fid100344RRef AS FId100344RRef, T15.Fid100344RRef,
       T15.Fid100343RRef AS FId100343RRef, T15.Fid100346 AS FId100346_,
       T15.Fid100345 AS FId100345_
FROM (SELECT T3.Fid100343RRef AS FId100343RRef,
       T3.Fid100344RRef AS FId100344RRef, MAX(T3._Period) AS MAXPERIOD_
FROM _InfoRg100342 T3
WHERE ((T3._Fld2488 = CAST(0 AS NUMERIC))) AND (T3._Period <=
       '2023-12-04 12:49:02'::timestamp AND (EXISTS(SELECT 1
FROM pg_temp.tt3319 T4
LEFT OUTER JOIN _Reference101 T5
ON (T4._Q_001_F_007RRef = T5._IDRRRef) AND (T5._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference395 T6
ON (T4._Q_001_F_008RRef = T6._IDRRRef) AND (T6._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference366I1 T7
ON (T4._Q_001_F_006RRef = T7._IDRRRef) AND (T7._Fld2488 = CAST(0 AS NUMERIC))
WHERE ((1=1 OR 1=0) AND (T4._Q_001_F_001RRef =
       '\216@030f\332\261R\333\021\356\222\212\342\004z\226'::bytea) AND
       (T5._Fld53293RRef = '\214\2160\271\351\244\310\0008<\227\003{-\356e'::bytea)
       AND ((T7._Fld59861RRef <= '\201Fnu\257Q\301Fp\027Q\211\214\001{::bytea)
       OR (T7._Fld59861RRef = '\201Fnu\257Q\301Fp\027Q\211\214\001{::bytea)))
       AND ((T3.Fid100343RRef = T5._Fld53293RRef) AND
       (T3.Fid100344RRef = T6._Fld101340RRef))
UNION ALL SELECT 1
FROM pg_temp.tt3319 T8
LEFT OUTER JOIN _Reference101 T9
ON (T8._Q_001_F_007RRef = T9._IDRRRef) AND (T9._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference101 T10
ON (T9._Fld53293RRef = T10._IDRRRef) AND (T10._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference395 T11
ON (T8._Q_001_F_008RRef = T11._IDRRRef) AND (T11._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference366I1 T12
ON (T8._Q_001_F_006RRef = T12._IDRRRef) AND (T12._Fld2488 = CAST(0 AS NUMERIC))
WHERE ((1=1 OR 1=0) AND (T8._Q_001_F_001RRef =
       '\216@030f\332\261R\333\021\356\222\212\342\004z\226'::bytea) AND
       (T9._Fld53293RRef = '\214\2160\271\351\244\310\0008<\227\003{-\356e'::bytea)
```

one query out of 115 502 – and not the longest one

One production ERP database:

- 115 502 distinct queryids
- query text: 33 – 126 000 chars
- execution time: 1 ms – 26 s
- calls: 1 – 5 075
- blocks read: 0 – 3.5 M (≈27 GB)
- plan nodes: 1 – 195
- *never executed* nodes: 0 – 52

Which one costs you money? Not the text, not the length, not the call count.



Closing the blind spot in query statistics

pg_stat_statements

calls	[10 000 000]
mean_exec_time	[100.0]
total_exec_time	[1 000 000 000]

278 hours of execution time,
near the top of any ranking by
total time.

pg_track_optimizer

nexecs	[10 000 000]
avg_error	[8.4]
f_join_filter	[0.74]

A lot of this query's time went
into rows a join built and then
threw away.

Same query. One tool measures how long it ran — the other,
whether it had to.



Assessing the instance's potential

- 1 Estimation error (rows estimated / rows actual)
is it worth manipulating table statistics?
- 2 Selectivity of scans and joins (rows removed by filter)
do we have enough indexes?
- 3 SubPlans' share of the execution time
is it time to rewrite the queries?
- 4 Spilling of intermediate results
should queries get more memory?
- 4 Plan stability
would a generic plan be a better idea?



Postgres extension pg_track_optimizer at a glance

executor
finishes a query

→
estimated vs actual
rows, per node

→
running stats
in shared memory

→
one view:
pg_track_optimizer

11

metrics per query [ID]

0 restarts

to start tracking: shared
memory via the DSM
registry, no preload needed

QueryID

Modified Query ID to
reduce the number of
different entries

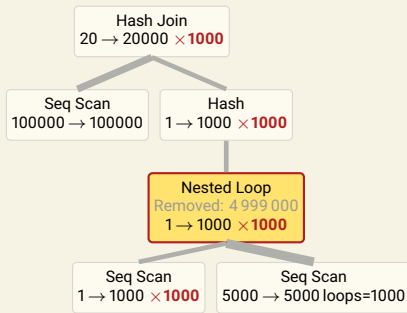
Persistent

flushed to disk on backend
exit, loaded back after a
restart



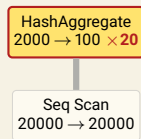
Nuance 1: Queries are different

Two joins over a filtered scan



6 nodes · 2 joins · 226 ms

GROUP BY over a scan



2 nodes · 0 joins · 1.4 ms

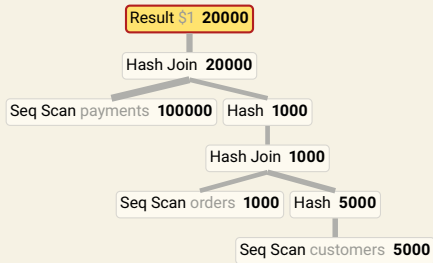
We should estimate optimisation potential of very different queries



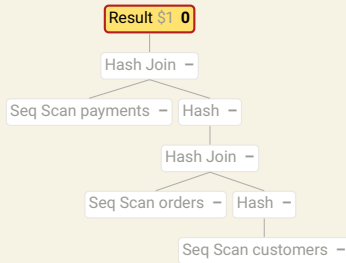
Nuance 2: Parameter might be in undefined state

```
PREPARE q(int, bool) AS  
  SELECT o.id, p.id FROM orders o  
    JOIN customers c ON c.id = o.cust JOIN payments p ON p.cust = c.id  
  WHERE $1;
```

EXECUTE q(true)



EXECUTE q(false)



Nuance 3: statistics' stochastic nature

```
ANALYZE t; -- 40 times, the data never change
EXPLAIN SELECT * FROM t WHERE x = 77; -- actual: 10000 rows every time
```

77 made it into the MCV list

$\approx 10\,000$ rows

Bitmap Heap Scan

21 runs of 40

77 missed the MCV list

≈ 60 rows, **off by $\times 170$**

Index Scan

19 runs of 40



Ratios, and how they spread

1 shapes and scales differ

2 nodes may not run

3 estimates drift

Dimensionless metrics

per plan node:

$$e = \left| \ln \frac{\text{actual}}{\text{estimated}} \right|$$

averaged over the nodes that *ran*;
weights: none, square, share of time,
share of cost

avg rms twa wca

rstats type

a distribution, not a number:

(count, mean, min, max, stddev)

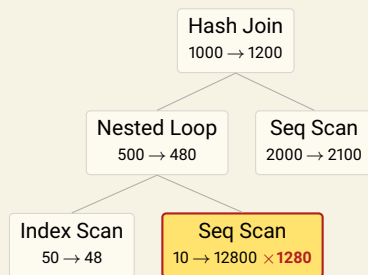
updated in place on every execution
(Welford), $O(1)$ memory

wide min..max or large stddev: the
plan is unstable, not just wrong

Any two queries become comparable —
and each one tells you how steady it is.



Cardinality estimation error



per-node error:

$$e = \left| \ln \frac{actual}{estimated} \right|$$

How to average across plan nodes?



How does it work in practice?

Test bed: **the Join Order Benchmark**

Leis et al., *How Good Are Query Optimizers, Really?*, PVLDB 2015

113

queries over real IMDb
data

17

tables joined in the
deepest query

Scans

many of them, including
parameterised ones

Room

plenty of it, for the planner
to go wrong



How the estimation error criteria work

TOP-10 by average error V/S TOP-10 by time-weighted error

```
SELECT avg.* FROM
  (SELECT queryid, query, avg_avg, twa_avg FROM pg_track_optimizer
   ORDER BY avg_avg DESC LIMIT 10) AS avg
JOIN
  (SELECT queryid FROM pg_track_optimizer
   ORDER BY twa_avg DESC LIMIT 10) AS twa
USING (queryid);
```

queryid	query	avg_avg	twa_avg
-6836119621424365027	/* 28a.sql */	3.91	2.16
3631768377448547020	/* 29c.sql */	3.77	2.51
-8544699698054799639	/* 22d.sql */	3.48	2.09
4682104415514032107	/* 7c.sql */	3.48	2.18

(4 rows)



A look inside 28a.sql

12.	0.000	0.000	↓ 15,866.0		1	→ Nested Loop (cost=673.574,16..713,608.92 rows=1 width=65) (actual rows=15,866.00 loops=1)
13.	0.000	0.000	↓ 2,641,497.5		1	→ Parallel Hash Join (cost=673.573,73..713,604.75 rows=2 width=69) (actual rows=5,282,995.00 loops=1) Hash Cond: (mk.movie_id = t.id)
14.	0.000	0.000	↓ 4.0		1	→ Parallel Seq Scan on movie_keyword mk (cost=0.00..35,789.82 rows=1,130,982 width=8) (actual rows=4,523,930.00 loops=1)
15.	0.000	0.000	↓ 47,805.0		1	→ Parallel Hash (cost=673.573,72..673,573.72 rows=1 width=61) (actual rows=47,805.00 loops=1) Buckets: 65,536 (originally 1024) Batches: 1 (originally 1) Memory Usage: 5,368kB
16.	0.000	0.000	↓ 47,805.0		1	→ Nested Loop (cost=611,718.06..673,573.72 rows=1 width=61) (actual rows=47,805.00 loops=1) Join Filter: (t.id = mk.movie_id)
17.	0.000	0.000	↓ 18,042.0		1	→ Nested Loop (cost=611,717.63..673,566.96 rows=3 width=40) (actual rows=54,126.00 loops=1)
18.	0.000	0.000	↓ 7,874.1		1	→ Parallel Hash Join (cost=611,717.20..673,551.17 rows=7 width=28) (actual rows=55,119.00 loops=1) Hash Cond: (pi.person_id = ci.person_id)
19.	0.000	0.000	↓ 94.7		1	→ Parallel Hash Join (cost=1.84..61,811.22 rows=6,553 width=4) (actual rows=620,526.00 loops=1) Hash Cond: (pi.info_type_id = it3.id)
20.	0.000	0.000	↓ 4.0		1	→ Parallel Seq Scan on person_info pi (cost=0.00..59,792.74 rows=740,474 width=8) (actual rows=2,963,664.00 loops=1)
21.	0.000	0.000	↑ 1.0		1	→ Parallel Hash (cost=1.83..1.83 rows=1 width=4) (actual rows=1.00 loops=1) Buckets: 1,024 Batches: 1 Memory Usage: 40kB
22.	0.000	0.000	↑ 1.0	- 112	1	→ Parallel Seq Scan on info_type it3 (cost=0.00..1.83 rows=1 width=4) (actual rows=1.00 loops=1) Filter: ((info)::text = 'trivia'::text) Rows Removed by Filter: 112

```

-> Parallel Hash Join (rows=6553) (actual rows=620526)
    Hash Cond: (pi.info_type_id = it3.id)
-> Parallel Seq Scan on person_info pi (rows=740474) (actual rows=2963664)
-> Parallel Hash (rows=1) (actual rows=1.00)
    -> Parallel Seq Scan on info_type it3 (rows=1) (actual rows=1)
        Filter: ((info)::text = 'trivia'::text)
        Rows Removed by Filter: 112

```

⇒ The x4 scan error grows throughout the join tree.

